

DataLytica

Master Build Prompt for Claude Code

Local-First AI Data Analytics Desktop App

Stack	Python + Electron + SQLite
AI Layer	DeepSeek API + Ollama (local)
Phases	14 build phases
License	HMAC-SHA256 offline key system
Output	Windows .exe installer

Paste this entire document into Claude Code to scaffold the full project.

How to Use This Prompt

Open your terminal, create an empty folder, and launch Claude Code inside it. Paste this entire document as your first message. Claude Code will build the project phase by phase.

Recommended: Do NOT ask Claude Code to complete all 14 phases at once. Work phase by phase — run the app after each phase and verify it works before proceeding.

Checkpoint Strategy

- After Phase 1: verify the folder structure exists on disk
- After Phase 3: test DeepSeek API connectivity with a sample call
- After Phase 4: drag a real CSV in and confirm data lands in SQLite
- After Phase 7: verify EDA stats are real pandas numbers, not AI-guessed
- After Phase 10: test license key generation and validation
- After Phase 12: run the full .exe installer end to end

Project Overview

Build a desktop data analytics application called DataLytica — a local-first, AI-powered alternative to Tableau and Power BI. The entire app runs offline after installation. No cloud database. No subscription backend. Revenue comes from selling offline license keys.

Layer	Technology
Backend	Python 3.11+ with FastAPI (local HTTP server on port 8765)
Frontend	Electron.js — renders local web UI as a native .exe application
Database	SQLite via sqlite3 + SQLAlchemy — stored at ~/.datalytica/datalytica.db
Data Engine	pandas, scipy, openpyxl, plotly — all computation done in Python
AI (primary)	DeepSeek API — ultra-low cost, ~\$0.001 per 1K tokens
AI (fallback)	Ollama — 100% local, zero API cost, works with no internet
Packaging	PyInstaller (Python → .exe) + electron-builder (full installer)
Licensing	HMAC-SHA256 offline key validation — no server required

PHASE 1 Project Scaffold

Create the following folder structure exactly:

```

datalytica/
├── backend/
│   ├── main.py                # FastAPI app entry point
│   ├── database.py            # SQLite connection + SQLAlchemy setup
│   ├── models.py              # SQLAlchemy ORM models
│   ├── routers/
│   │   ├── ingest.py          # File upload + schema detection
│   │   ├── cleanse.py          # Data cleaning endpoints
│   │   ├── etl.py              # Transform/reshape endpoints
│   │   ├── eda.py              # EDA stats endpoints
│   │   ├── charts.py           # Chart generation endpoints
│   │   ├── narrative.py        # AI report generation endpoints
│   │   └── license.py          # License key validation
│   ├── services/
│   │   ├── ai_service.py       # DeepSeek + Ollama abstraction
│   │   ├── data_service.py     # pandas pipeline logic
│   │   ├── chart_service.py    # Plotly chart builder
│   │   ├── eda_service.py      # Statistical analysis
│   │   ├── cleanse_service.py  # Cleaning logic
│   │   ├── etl_service.py      # ETL transforms
│   │   └── license_service.py   # Key generation + validation
│   └── requirements.txt
├── frontend/
│   ├── package.json
│   ├── main.js                # Electron main process
│   ├── preload.js             # Electron preload script
│   ├── renderer/
│   │   ├── index.html         # App shell
│   │   ├── styles/
│   │   │   └── main.css        # Design system styles
│   │   └── pages/
│   │       ├── dashboard.html
│   │       ├── ingest.html
│   │       ├── cleanse.html
│   │       ├── etl.html
│   │       ├── eda.html
│   │       ├── charts.html
│   │       ├── narrative.html
│   │       └── settings.html
├── installer/
│   ├── build.py
│   └── datalytica.spec
└── README.md

```

After creating this structure, initialize:

- backend/requirements.txt with all Python dependencies
- frontend/package.json with all Node dependencies
- backend/database.py with SQLite path at ~/.datalytica/datalytica.db

PHASE 2**Database & Models**

In backend/database.py: Connect SQLite to ~/.datalytica/datalytica.db (create dir if not exists). Use SQLAlchemy ORM with create_engine + declarative_base.

In backend/models.py, create these five tables:

Dataset

Stores metadata for each uploaded file.

```
id: int (PK)
name: str
original_filename: str
row_count: int
column_count: int
file_size_kb: float
created_at: datetime
updated_at: datetime
status: str # 'raw', 'cleansed', 'transformed'
sqlite_table_name: str # where the actual data rows are stored
```

DatasetColumn

Column metadata per dataset.

```
id: int (PK)
dataset_id: int (FK)
column_name: str
detected_type: str # 'numeric', 'categorical', 'datetime', 'text', 'boolean'
null_count: int
null_pct: float
unique_count: int
sample_values: str # JSON array of 5 sample values
```

PipelineStep

Audit log of every transform applied — stores the actual Python code that ran.

```
id: int (PK)
dataset_id: int (FK)
step_type: str # 'cleanse', 'etl', 'filter', etc.
description: str
python_code: str # actual pandas code that was executed
applied_at: datetime
rows_before: int
rows_after: int
```

Report

Saved AI narrative reports.

```
id: int (PK)
dataset_id: int (FK)
title: str
content_markdown: str
created_at: datetime
```

AppConfig

Key-value store for application settings (API keys, license, AI provider).

```
key: str (PK)
value: str
```

PHASE 3**AI Service — Anti-Hallucination Core**

In backend/services/ai_service.py, create a class AIService with:

- provider: 'deepseek' or 'ollama' — read from AppConfig
- deepseek_api_key: read from AppConfig
- ollama_model: default 'mistral'
- Method complete(system_prompt, user_prompt) -> str

DeepSeek call: POST to <https://api.deepseek.com/v1/chat/completions> — model: deepseek-chat, temperature: 0.1

Ollama fallback: POST to <http://localhost:11434/api/generate> — if DeepSeek fails, auto-switch.

CRITICAL: The anti-hallucination system prompt below MUST be used on every AI call in this application. This is non-negotiable.

```
ANTI_HALLUCINATION_SYSTEM_PROMPT = """
You are a data analyst assistant. You ONLY report facts derived from the
computed statistics provided to you. You NEVER infer, guess, extrapolate,
or add any fact that is not explicitly present in the data JSON provided.
If a value is not in the data, say 'not available' — never make it up.
Every claim you make must reference a specific column name or metric from
the provided data. Do not use phrases like 'likely', 'probably', 'seems to',
or 'suggests' unless directly supported by a statistical test result.
"""
```

PHASE 4 Data Ingestion

In backend/routers/ingest.py and backend/services/data_service.py:

POST /api/ingest/upload

- Accept multipart file upload: CSV, XLSX, JSON, TSV
- Use pandas to read the file with auto-encoding detection via chardet
- Auto-detect column types: try `pd.to_numeric`, `pd.to_datetime`, count unique values
- If unique < 20 and < 5% of rows → categorical. Otherwise → text
- Store DataFrame into SQLite as table named `data_{uuid4().hex[:8]}`
- Save Dataset + DatasetColumn records to database
- Return: `dataset_id`, `name`, `row_count`, `column_count`, `columns`, and first 10 rows preview

GET /api/ingest/preview/{dataset_id}

- Return first 100 rows of the dataset as JSON

POST /api/ingest/join

- Accept: `dataset_id_1`, `dataset_id_2`, `join_column`, `join_type`
- Perform pandas merge, save as new dataset
- Return new dataset metadata

PHASE 5**AI Data Cleansing**

In backend/services/cleanse_service.py and backend/routers/cleanse.py:

POST /api/cleanse/scan/{dataset_id}

Returns a full cleansing report — computed entirely by pandas, NOT by AI:

```
{
  "null_report": [{"column": "...", "null_count": N, "null_pct": X}],
  "duplicate_rows": N,
  "outlier_report": [{"column": "...", "outlier_count": N, "method": "IQR"}],
  "type_mismatch": [{"column": "...", "issue": "..."}],
  "format_issues": [{"column": "...", "issue": "mixed date formats"}]
}
```

Then call AIService with this JSON + anti-hallucination system prompt to generate plain-English explanations only. AI explains — pandas detected.

POST /api/cleanse/apply

Accept: dataset_id and operations array. Each operation:

```
{
  "type": "drop_nulls" | "fill_mean" | "fill_median" | "fill_value"
    | "drop_duplicates" | "remove_outliers" | "standardize_format",
  "column": "column_name",
  "params": {}
}
```

- Apply each operation with pandas
- Log the generated Python code into PipelineStep
- Return: rows_before, rows_after, operations_applied, python_code

PHASE 6**ETL — Transform & Reshape**

In backend/services/etl_service.py and backend/routers/etl.py:

POST /api/etl/transform — Natural Language ETL

Accept: dataset_id and instruction (natural language string). Examples:

- "Create a new column profit = revenue - cost"
- "Pivot the table with date as index and category as columns"
- "Filter rows where sales > 1000"
- "Group by region and sum revenue"

Process:

1. Load dataset schema (column names + types) from DatasetColumn
2. Send to AIService with code-generation system prompt:
'Given DataFrame df with columns {columns},
generate ONLY Python pandas code for: {instruction}.
Return ONLY valid Python. No explanations. No imports.
Result must be stored back in df.'
3. Execute generated code safely via exec() in sandboxed namespace
containing only df and pd
4. Validate result is still a DataFrame
5. Save result + log PipelineStep with the generated code
6. Return: python_code, rows_before, rows_after, first 10 rows preview

IMPORTANT: Always show the generated Python code to the user BEFORE execution. Never run silently.

Explicit ETL Endpoints (no AI needed)

- POST /api/etl/pivot — pandas pivot_table
- POST /api/etl/aggregate — groupby + agg
- POST /api/etl/filter — query string filter
- POST /api/etl/add_column — formula-based new column
- POST /api/etl/rename — rename columns
- POST /api/etl/export — export to CSV or XLSX

PHASE 7 Exploratory Data Analysis (EDA)

In backend/services/eda_service.py and backend/routers/eda.py:

GET /api/eda/full/{dataset_id}

Compute entirely in pandas/scipy — NO AI for computation. Returns:

```
{
  "summary": { "row_count": N, "column_count": N, "memory_mb": X, "duplicate_rows":
N },
  "numeric_stats": {
    "column_name": {
      "mean": X, "median": X, "std": X, "min": X, "max": X,
      "q25": X, "q75": X, "skewness": X, "kurtosis": X,
      "null_count": N, "null_pct": X
    }
  },
  "categorical_stats": {
    "column_name": {
      "unique_count": N, "top_value": "...", "top_freq": N,
      "value_counts": { "val": count },
      "null_count": N
    }
  },
  "correlations": {
    "matrix": [[...]], "columns": [...],
    "top_pairs": [{ "col1": "...", "col2": "...", "r": X }]
  },
  "distributions": { "column_name": { "bins": [...], "counts": [...] } }
}
```

Then call AIService with the full stats JSON + anti-hallucination prompt:

```
"Given these computed statistics, write exactly 5 insight statements.
Each statement MUST cite a specific column name and a specific numeric
value from the stats JSON. Do not add any fact not present in the JSON.
Format: numbered list, one sentence each, plain English."
```

Return both raw stats AND AI insights in one response.

PHASE 8**Charts & Visualization**

In backend/services/chart_service.py and backend/routers/charts.py:

POST /api/charts/recommend

- Input: dataset_id, question (natural language)
- Load column metadata from DatasetColumn
- Ask AI: 'Given columns {columns_with_types}, what is the best chart type to answer: {question}?'
- Return JSON: chart_type, x_column, y_column, color_column (optional), reason

POST /api/charts/generate

Input: dataset_id, chart_type, x_column, y_column, color_column, title, filters

Supported chart types via Plotly:

```
bar      → px.bar
line     → px.line
scatter  → px.scatter
pie      → px.pie
histogram → px.histogram
heatmap  → px.density_heatmap
box      → px.box
area     → px.area
```

- Return Plotly figure as JSON via fig.to_json()
- Also return the Python code that generated the chart

PHASE 9**AI Narrative Report**

In backend/routers/narrative.py:

POST /api/narrative/generate

Input: dataset_id, report_title, tone ('executive' | 'technical' | 'simple'), focus_columns

Process: First run full EDA (Phase 7) to get computed stats. Then build this structured prompt:

```
SYSTEM = ANTI_HALLUCINATION_SYSTEM_PROMPT + '''
You are a business report writer. Write in {tone} tone.
Structure:
  1. Executive Summary (2-3 sentences)
  2. Key Findings (5 bullet points, each citing a specific number)
  3. Notable Patterns (2-3 paragraphs)
  4. Recommendations (3 actionable items based only on the data)

STRICT RULES:
- Every number you write must appear in the stats JSON below
- Never write a sentence without a data reference
- Forbidden words: 'likely', 'probably', 'may', 'might', 'could suggest'
- If you cannot support a claim with the data, omit it
'''

USER = f'Dataset: {dataset_name}\nStats:\n{json.dumps(eda_stats, indent=2)}'
```

POST /api/narrative/export

- Input: report_id, format ('pdf' | 'docx' | 'markdown')
- PDF: use reportlab
- DOCX: use python-docx
- Return file as download response

PHASE 10**License Key System**

In backend/services/license_service.py and backend/routers/license.py:

Change the SECRET string before shipping. This is your master key — keep it private and never commit it to git.

```
import hmac, hashlib, platform, uuid, base64, json

SECRET = "DATALYTICA_SECRET_2025_CHANGE_THIS"

def get_machine_id() -> str:
    """Fingerprint: MAC address + platform + node name, hashed"""
    raw = f"{uuid.getnode()}-{platform.system()}-{platform.node()}"
    return hashlib.sha256(raw.encode()).hexdigest()[:16]

def generate_key(machine_id: str, plan: str, expiry: str) -> str:
    """Generate a license key for a machine — run on your own machine"""
    payload = json.dumps({"m": machine_id, "p": plan, "e": expiry})
    sig = hmac.new(SECRET.encode(), payload.encode(),
                   hashlib.sha256).hexdigest()[:16]
    b64 = base64.b64encode(payload.encode()).decode()
    return f"DL-{b64[:20]}-{sig}".upper()

def validate_key(key: str) -> dict:
    """Validate key offline — no server required"""
    try:
        parts = key.upper().split('-')
        payload = base64.b64decode(parts[1] + '==').decode()
        data = json.loads(payload)
        expected_sig = hmac.new(SECRET.encode(), payload.encode(),
                                hashlib.sha256).hexdigest()[:16]
        if not hmac.compare_digest(parts[2], expected_sig.upper()):
            return {"valid": False, "error": "Invalid key"}
        if get_machine_id() != data['m']:
            return {"valid": False, "error": "Wrong machine"}
        from datetime import datetime
        if datetime.now().strftime('%Y-%m-%d') > data['e']:
            return {"valid": False, "error": "License expired"}
        return {"valid": True, "plan": data["p"], "expiry": data["e"]}
    except Exception as e:
        return {"valid": False, "error": str(e)}
```

License Endpoints

- GET /api/license/machine-id — return current machine fingerprint (user sends this to you to generate a key)
- POST /api/license/validate — validate + store license in AppConfig
- GET /api/license/status — return current license validity + plan + expiry

Add middleware that checks license on every API request. If invalid or expired, return HTTP 403 with `{"error": "license_required"}`.

PHASE 11 Electron Frontend

In frontend/main.js:

- Spawn Python FastAPI backend as child process on port 8765
- Poll GET /api/health every 500ms until 200 response
- Show splash screen during startup
- Open BrowserWindow loading renderer/index.html
- On window close, kill the Python child process

UI Structure

- Left sidebar navigation: Dashboard, Datasets, Cleanse, ETL, EDA, Charts, Reports, Settings
- Top bar: dataset selector dropdown + license status badge
- Main content area loads different pages per nav item

Design System (styles/main.css)

```
--primary: #534AB7
--primary-dark: #3C3489
--surface: #F8F8F8
--border: #E2E2E2
--text: #333333
--text-muted: #888888

Font: system-ui / Inter
Dark mode: supported via prefers-color-scheme
Components: cards, tables, buttons (primary, secondary, danger), badges
Charts: rendered via Plotly.react() from JSON returned by backend
```

Page-by-Page UI Requirements

- Ingest: drag-drop zone + file picker + schema preview table + multi-file join wizard
- Cleanse: issues list with severity badges + diff preview + one-click apply per issue
- ETL: natural language input + generated code preview (syntax highlighted) + before/after row count
- EDA: stats cards grid + correlation heatmap (Plotly) + distributions + AI insights panel
- Charts: column picker (drag X/Y) + chart type selector + live Plotly preview + AI recommend button
- Narrative: tone selector + generate button + markdown preview + export PDF/DOCX buttons
- Settings: AI provider toggle (DeepSeek/Ollama) + API key input + license key entry + machine ID display

All pages communicate with the Python backend via `fetch('http://localhost:8765/api/...')`.

PHASE 12**Build & Packaging****PyInstaller Spec (installer/datalytica.spec)**

```
Entry point: backend/main.py
Mode: onedir (not onefile — faster startup)
Name: datalytica-backend
Hidden imports: uvicorn, fastapi, sqlalchemy, pandas, scipy, plotly, openpyxl
Include: entire backend/ directory
```

electron-builder Config (frontend/package.json)

```
{
  "build": {
    "appId": "com.datalytica.app",
    "productName": "DataLytics",
    "win": { "target": "nsis", "icon": "assets/icon.ico" },
    "extraResources": [
      { "from": "../dist/datalytica-backend/", "to": "backend/" }
    ],
    "nsis": {
      "oneClick": false,
      "allowToChangeInstallationDirectory": true
    }
  }
}
```

Build Script (installer/build.py)

```
# Run in order:
# 1. pip install -r backend/requirements.txt
# 2. PyInstaller: pyinstaller installer/datalytica.spec
# 3. npm install in frontend/
# 4. electron-builder: npx electron-builder --win
```

PHASE 13**Dependencies****backend/requirements.txt**

```
fastapi==0.111.0
uvicorn==0.29.0
sqlalchemy==2.0.30
pandas==2.2.2
scipy==1.13.0
numpy==1.26.4
openpyxl==3.1.2
plotly==5.22.0
python-multipart==0.0.9
chardet==5.2.0
httpx==0.27.0
python-docx==1.1.0
reportlab==4.2.0
aiofiles==23.2.1
```

Frontend CDN Libraries (no bundler needed)

```
Plotly.js:  https://cdn.plot.ly/plotly-2.32.0.min.js
Marked.js:  https://cdn.jsdelivr.net/npm/marked/marked.min.js
Highlight:  https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js
```

PHASE 14 **Startup Sequence**

Implement this exact user startup flow:

1	User double-clicks DataLytica.exe
2	Electron main.js spawns datalytica-backend.exe on port 8765
3	Electron polls GET /api/health every 500ms — shows splash screen
4	On first 200 response, check license via GET /api/license/status
5	If no valid license — show license entry screen with machine ID displayed
6	If valid license — load Dashboard showing all saved datasets
7	User drags and drops a CSV file to immediately begin analysis

Implementation Notes for Claude Code

Read all these rules before starting. They prevent the most common build failures.

- Build and test each phase before moving to the next — never skip ahead
- After Phase 3 (AI Service), test DeepSeek connectivity with a sample call before proceeding
- After Phase 4 (Ingest), test with a real CSV file to confirm SQLite storage works
- The anti-hallucination system prompt is NON-NEGOTIABLE — every AI call must use it
- Generated Python code from ETL (Phase 6) must be shown to the user before execution — never run silently
- All pandas operations must handle empty DataFrames and NaN values gracefully
- SQLite table names must be sanitized — alphanumeric and underscore only
- License check middleware must be the FIRST thing that runs on every route
- Never hardcode the SECRET key — load it from an environment variable in production

First Command After Scaffold

After Phase 1 is complete, run:

```
cd backend
pip install -r requirements.txt
python main.py
```

Then verify `http://localhost:8765/api/health` returns `{"status": "ok"}`. If it does, the foundation is working. Proceed phase by phase from there.

You are now ready to paste this document into Claude Code. Good luck building DataLytics!